

03 - Classifier Evaluation and Comparison

Alessandro Bregoli

2025

Introduction

In this tutorial we will **describe** how to **evaluate** and **compare** a **classifier**.



Introduction

In this tutorial we will **describe** how to **evaluate** and **compare** a **classifier**.

Specifically, we will deepen the following concepts:

Classification Error

- Train Error
- Validation Error

Model Validation

- Holdout
- Cross Validation

Confidence Interval

- Model performance
- Model Comparison



Introduction

In this tutorial we will **describe** how to **evaluate** and **compare** a **classifier**.

Specifically, we will deepen the following concepts:

Classification Error

- Train Error
- Validation Error

Model Validation

- Holdout
- Cross Validation

Confidence Interval

- Model performance
- Model Comparison



Furthermore, we will introduce the concept of **flow variable** in **Knime**.

Classification errors

Validation - Confusion Matrix

The **evaluation** of a **classification model** must quantify how well the model is able to **predict the right class**.

Validation - Confusion Matrix

The **evaluation** of a **classification model** must quantify how well the model is able to **predict the right class**.

For a **binary classifier**, the **Confusion Matrix** counts the number of correct and wrong prediction as follows:

		Predicted Class	
		False	True
Actual Class	False	TN	FP
	True	FN	TP

- **TP**: True Positive
- **TN**: True Negative
- **FP**: False Positive
- **FN**: False Negative

Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

-
-
-
-

Scorer



Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

- **Precision**

-
-
-

Precision measures the proportion of **positive predictions** that were **actually correct**.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Scorer



Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

- Precision
- **Recall**
-
-

Recall measures the **proportion of actual positive cases** that were **correctly identified**.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Scorer



Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

- Precision
- Recall
- **F1**
-

F1 is the **harmonic mean** of **precision** and **recall**.

$$F1 = \frac{2}{\text{Precision}^{-1} + \text{Recall}^{-1}}$$

Scorer



Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

- Precision
- Recall
- F1
- **Accuracy**

Accuracy measures the **proportion** of cases **correctly classified**.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Scorer



Classification Error

Regardless of the selected metric, when you validate a model you must take into account:

-
-

Classification Error

Regardless of the selected metric, when you validate a model you must take into account:

- **Training Performance:** the performance observed on the training set.
-

Classification Error

Regardless of the selected metric, when you validate a model you must take into account:

- **Training Performance:** the performance observed on the training set.
- **Generalization Performance:** the expected performance on previously unseen records (test set).

Classification Error

Regardless of the selected metric, when you validate a model you must take into account:

- **Training Performance:** the performance observed on the training set.
- **Generalization Performance:** the expected performance on previously unseen records (test set).

A **good classification model** must obtain high performance both in train and in test:

Classification Error

Regardless of the selected metric, when you validate a model you must take into account:

- **Training Performance:** the performance observed on the training set.
- **Generalization Performance:** the expected performance on previously unseen records (test set).

A **good classification model** must obtain high performance both in train and in test:

- A model with **poor training performance** suffer from **underfitting**
-

Classification Error

Regardless of the selected metric, when you validate a model you must take into account:

- **Training Performance:** the performance observed on the training set.
- **Generalization Performance:** the expected performance on previously unseen records (test set).

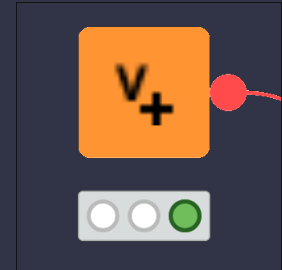
A **good classification model** must obtain high performance both in train and in test:

- A model with **poor training performance** suffer from **underfitting**
- A model with **high training performance** but **low generalization performance** suffer from **overfitting**.

Flow Variables

Flow variables are parameters that can be used to **avoid manually changing settings** withing the **nodes** of a workflow.

Variable Creator

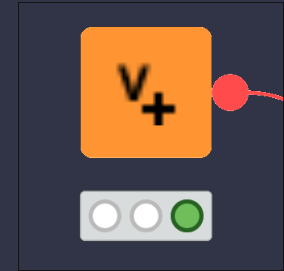


Flow Variables

Flow variables are parameters that can be used to **avoid manually changing settings** withing the **nodes** of a workflow.

The most direct way to create a flow variable is through the *Variable Creation* node.

Variable Creator

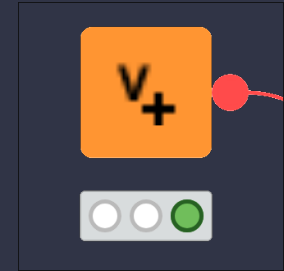


Flow Variables

Flow variables are parameters that can be used to **avoid manually changing settings** withing the **nodes** of a workflow.

The most direct way to create a flow variable is through the *Variable Creation* node.

Variable Creator



Variable Cretor Config

Variable Creator

Flow variable 1

Name

seed

Type

IntType

Value

2025

+

Add variable

Flow Variables

Flow variables are parameters that can be used to **avoid manually changing settings** withing the **nodes** of a workflow.

The most direct way to create a flow variable is through the *Variable Creation* node.

Variable Creator

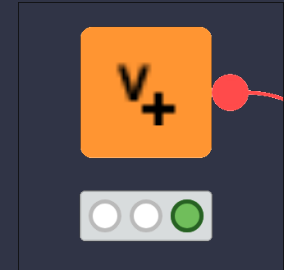
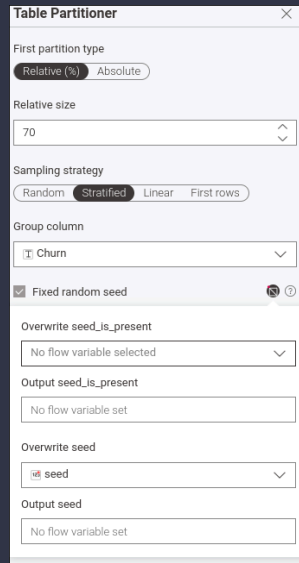


Table Partitioner Config (seed)

The image shows a screenshot of the 'Table Partitioner' configuration window. The window has a title bar with a close button. The configuration is organized into several sections: 'First partition type' with radio buttons for 'Relative (%)' (selected) and 'Absolute'; 'Relative size' with a text input field containing '70'; 'Sampling strategy' with radio buttons for 'Random', 'Stratified' (selected), 'Linear', and 'First rows'; 'Group column' with a dropdown menu showing 'Churn'; a checkbox for 'Fixed random seed' which is checked; 'Overwrite seed_is_present' with a dropdown menu showing 'No flow variable selected'; 'Output seed_is_present' with a text input field containing 'No flow variable set'; 'Overwrite seed' with a dropdown menu showing 'seed'; and 'Output seed' with a text input field containing 'No flow variable set'.

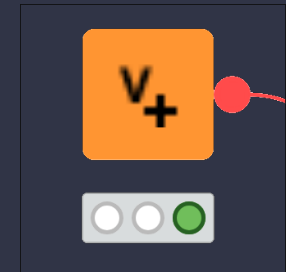
Flow Variables

Flow variables are parameters that can be used to **avoid manually changing settings** withing the **nodes** of a workflow.

The most direct way to create a flow variable is through the *Variable Creation* node.

RPropr MLP Learner Config (seed and number of neurons)

Variable Creator



Options		Flow Variables	Job Manager Selection
☐ maxiter			
☐ hiddenlayer			
☐ nrhiddenneurons	☐ neurons		
☐ classcol			
☐ ignoremv			
☐ useRandomSeed			
☐ randomSeed	☐ seed		

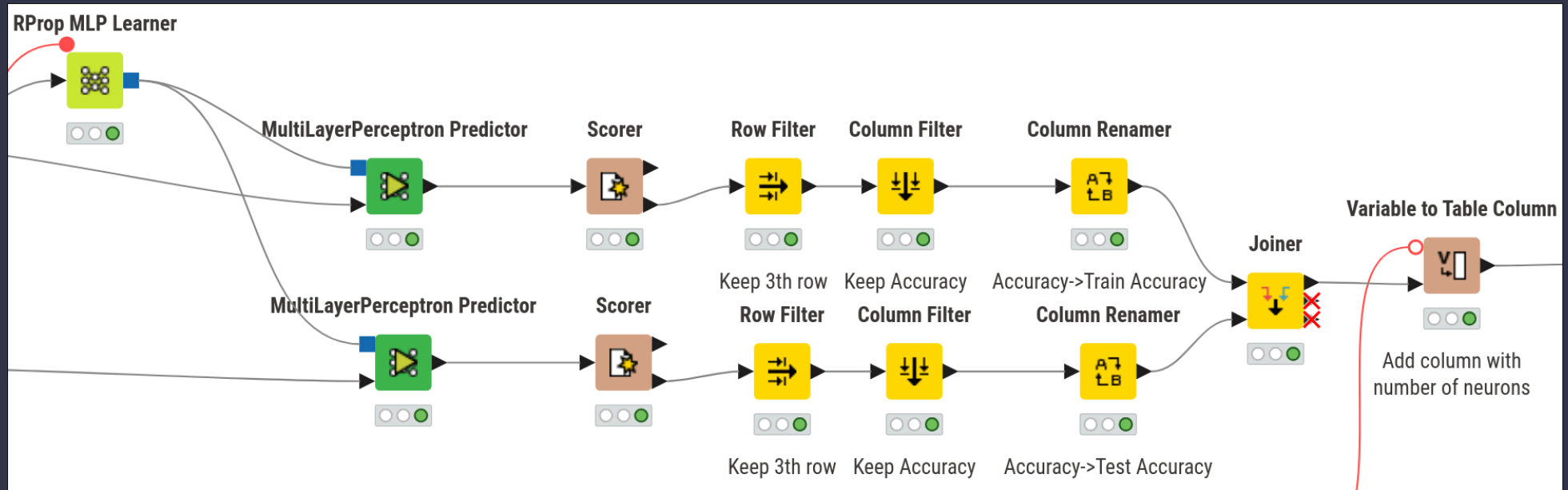
Hands on (45 minutes)

- **Import** the file **Churn_yn.xls**.
- **Remove** the columns **State**, **Phone** and **Area Code**.
- **Split** the dataset (70% train 30% test) and set the seed to **2025**.
- **Normalize** train and test using the method: **z-score**.
- **Train 4 RProp MLP Learners** with seed equal to **2025**, **1 hidden layer** and **1, 2, 4, 8 neurons** respectively.
- **Evaluate** the performance of each model in train and test using a **scorer** node.
- **Combine** the results and show them using a **line plot** node.
- When do we observe **underfitting/overfitting**?
- **Bonus:** use the flow variables for:
 - Seed (Table Partitioner)
 - randomSeed (RProp MLP Learner)
 - Number of hidden neurons (RProp MLP Learner)



Hands on (Hint)

For each model create 3 columns: **Train Accuracy**, **Test Accuracy** and **number of neurons**.



Model Validation

Introduction

The **Validation** of a **machine learning** model is a critical task.



Introduction

The **Validation** of a **machine learning** model is a critical task.

When we validate a model we want to access:

- Prediction accuracy
- Generalization accuracy
- Stability



Introduction

The **Validation** of a **machine learning** model is a critical task.

When we validate a model we want to access:

- Prediction accuracy
- Generalization accuracy
- Stability

Furthermore, we want to access the **confidence** of our **validation**.



Introduction

The **Validation** of a **machine learning** model is a critical task.

When we validate a model we want to access:

- Prediction accuracy
- Generalization accuracy
- Stability

Furthermore, we want to access the **confidence** of our **validation**.

The next slides will presents the most common validation procedures:

- **Holdout**
- **Iterated Holdout**
- **Cross Validation**



Holdout

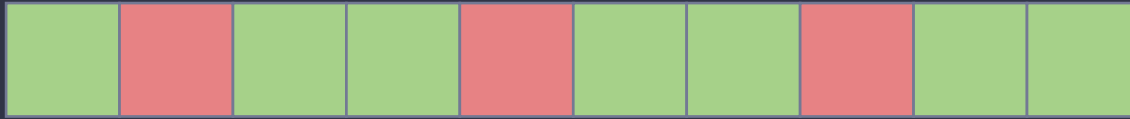
The **Holdout** method consists in splitting the dataset in two independent partitions:

Partitioner



Holdout

The **Holdout** method consists in splitting the dataset in two independent partitions:



where:

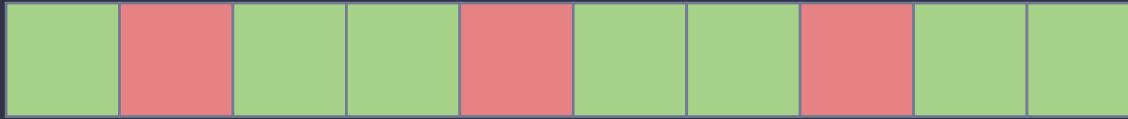
- **Green** boxes represent the train set.
- **Red** boxes represent the test set.

Partitioner



Holdout

The **Holdout** method consists in splitting the dataset in two independent partitions:



where:

- **Green** boxes represent the train set.
- **Red** boxes represent the test set.

The **holdout** is the approach used in all the previous workflows and can be obtained using the *Partitioner* node.

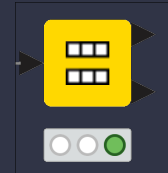
Partitioner



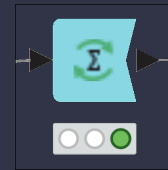
Iterated Holdout

The **Iterated Holdout** method consists in **iterating** the Holdout algorithm :

Partitioner



**Counting Loop
Start**

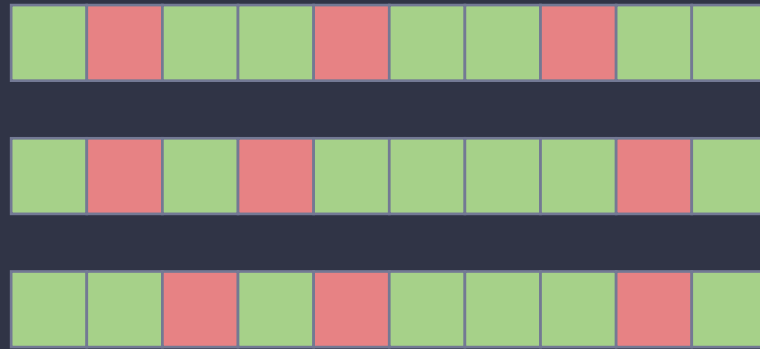


Loop End



Iterated Holdout

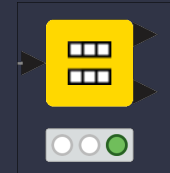
The **Iterated Holdout** method consists in **iterating** the Holdout algorithm :



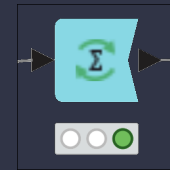
where:

- **Green** boxes represent the train set.
- **Red** boxes represent the test set.

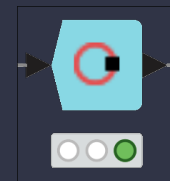
Partitioner



**Counting Loop
Start**

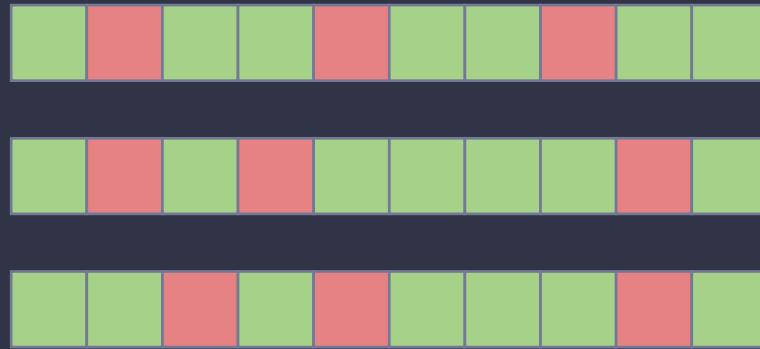


Loop End



Iterated Holdout

The **Iterated Holdout** method consists in **iterating** the Holdout algorithm :

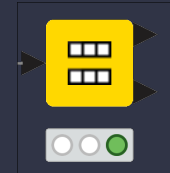


where:

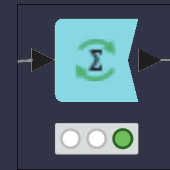
- **Green** boxes represent the train set.
- **Red** boxes represent the test set.

The **iterated holdout** can be obtained **surrounding** the **holdout** procedure using the nodes *Counting Loop Start* and *Loop End*

Partitioner



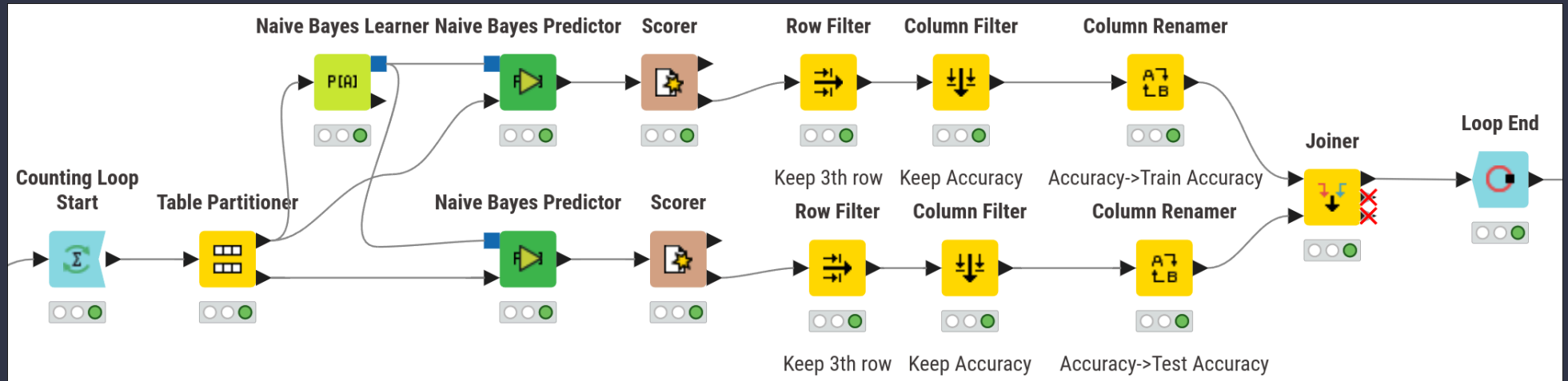
Counting Loop Start



Loop End



Iterated Holdout - Example

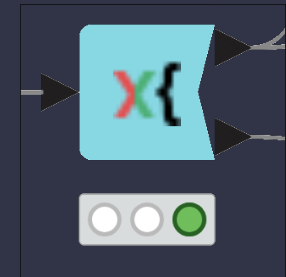


When performing this operation **do not** set the **seed** of the partitioner node to a **constant value**. Otherwise, all the iterations will split the dataset in the same way.

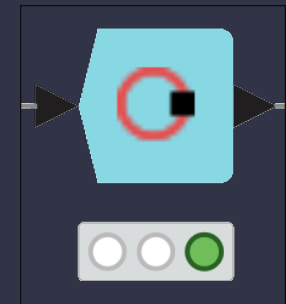
Cross Validation

The **Cross Validation** method split the dataset in K disjoint partitions (folds), then it test on each partition while training on the rest.

X-Partitioner

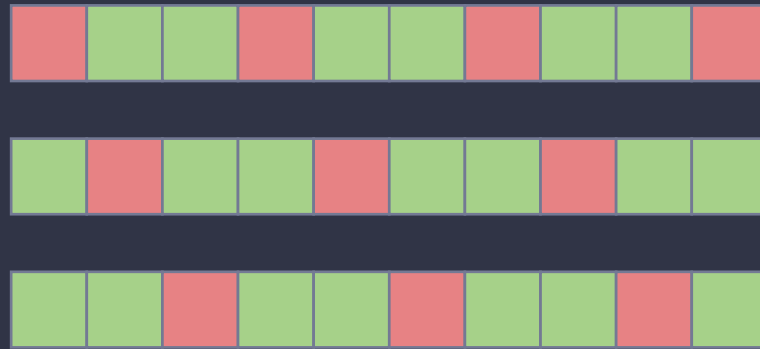


Loop End



Cross Validation

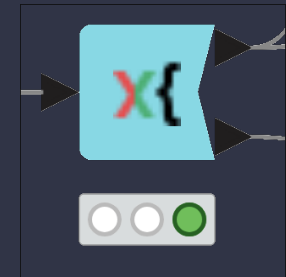
The **Cross Validation** method split the dataset in K disjoint partitions (folds), then it test on each partition while training on the rest.



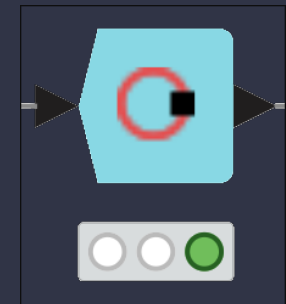
where:

- **Green** boxes represent the train set.
- **Red** boxes represent the test set.

X-Partitioner

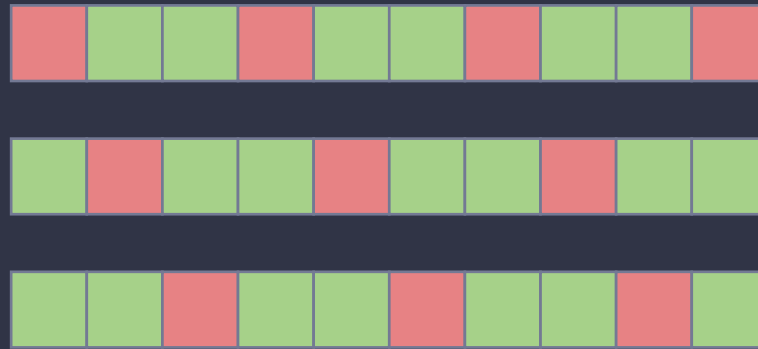


Loop End



Cross Validation

The **Cross Validation** method split the dataset in K disjoint partitions (folds), then it test on each partition while training on the rest.

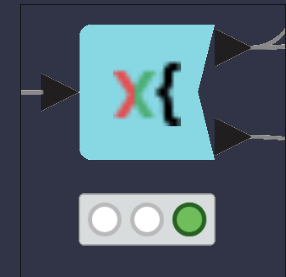


where:

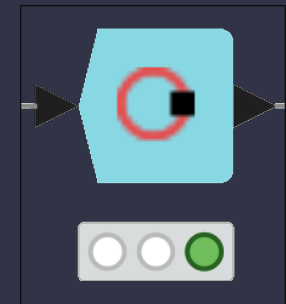
- **Green** boxes represent the train set.
- **Red** boxes represent the test set.

The **cross validation** can be carried out using the nodes *X-Partitioner* *Loop End*.

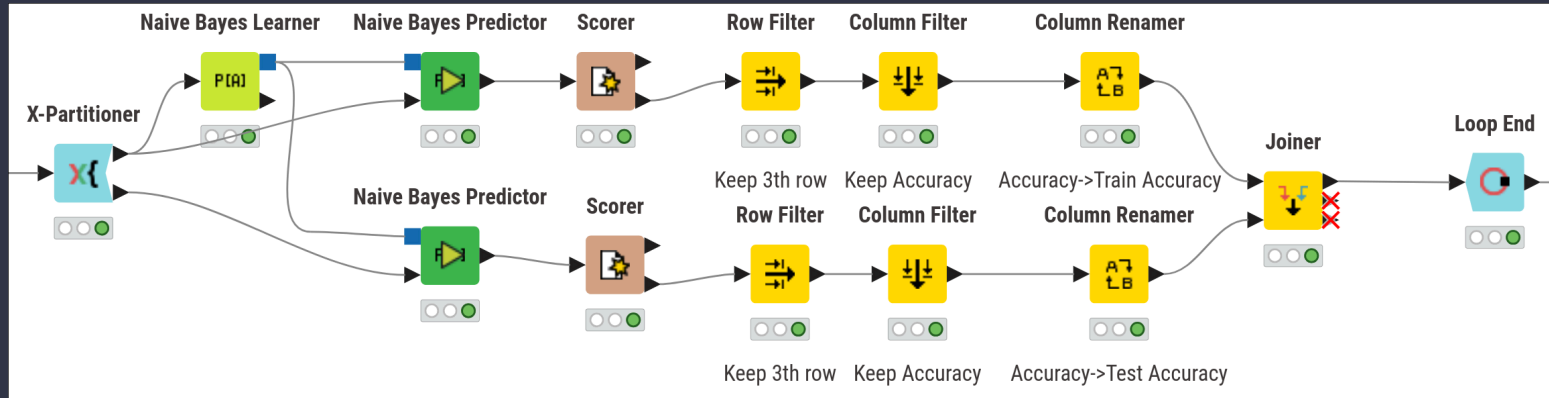
X-Partitioner



Loop End



Cross Validation - Example



X-Partitioner Config

Job Manager Selection	Memory Policy
Standard settings	
Flow Variables	
Number of validations	10
Linear sampling	☐
Random sampling	☐
Stratified sampling	☒
Class column	S Churn
☐ Random seed	0
Leave-one-out	☐

Hands on (15 minutes)

- **Load** the **Churn_yn.xls** dataset.
- **Remove** the columns **State**, **Phone** and **Area Code**.
- **Chose** one classifier.
- **Validate** it with:
 - Holdout
 - Iterated Holdout (10 Iterations)
 - *Hint: Use partitioner node in combination with the nodes Counting Loop Start and Loop End.*
 - Cross Validation (10 Iterations)
 - *Hint: Use the nodes X-Partitioner and Loop End.*
 - Leave One Out Cross Validation
 - *Hint: Use the nodes X-Partitioner and Loop End.*
- **Show** the results with a **Box Plot**.



Comparing Classifiers

Confidence Interval - Accuracy

Up to now we have **evaluated** the models using the **accuracy** estimated over the dataset.

Confidence Interval - Accuracy

Up to now we have **evaluated** the models using the **accuracy** estimated over the dataset.

How much the **estimated accuracy** is close to the **real** but **unknown** accuracy of the model?

Confidence Interval - Accuracy

Up to now we have **evaluated** the models using the **accuracy** estimated over the dataset.

How much the **estimated accuracy** is close to the **real** but **unknown** accuracy of the model?

To answer this question we can use the **Wilson Score Interval**:

$$\left(\frac{\text{acc} + \frac{z_{1-\frac{\alpha}{2}}^2}{2n} - \frac{z_{1-\frac{\alpha}{2}}}{2n} \sqrt{4n \cdot \text{acc} \cdot (1 - \text{acc}) + z_{1-\frac{\alpha}{2}}^2}}{1 + \frac{z_{1-\frac{\alpha}{2}}^2}{n}}, \frac{\text{acc} + \frac{z_{1-\frac{\alpha}{2}}^2}{2n} + \frac{z_{1-\frac{\alpha}{2}}}{2n} \sqrt{4n \cdot \text{acc} \cdot (1 - \text{acc}) + z_{1-\frac{\alpha}{2}}^2}}{1 + \frac{z_{1-\frac{\alpha}{2}}^2}{n}} \right)$$

where:

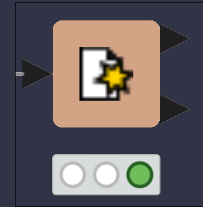
- **acc**: Is the estimated accuracy
- **n**: number of records in the dataset.
- $z_{1-\frac{\alpha}{2}}$: z-score for the confidence interval at $(1 - \alpha)$ confidence level

Confidence Interval with Knime - Accuracy

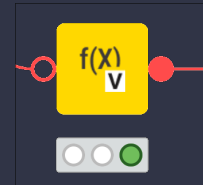
Knime does not have any node for computing the **Wilson Score Interval**.

Thus, we need to implement it using the *Expression/Variable Expression* node.

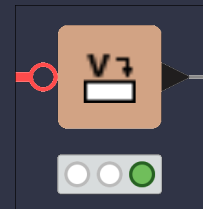
Scorer



Variable Expression



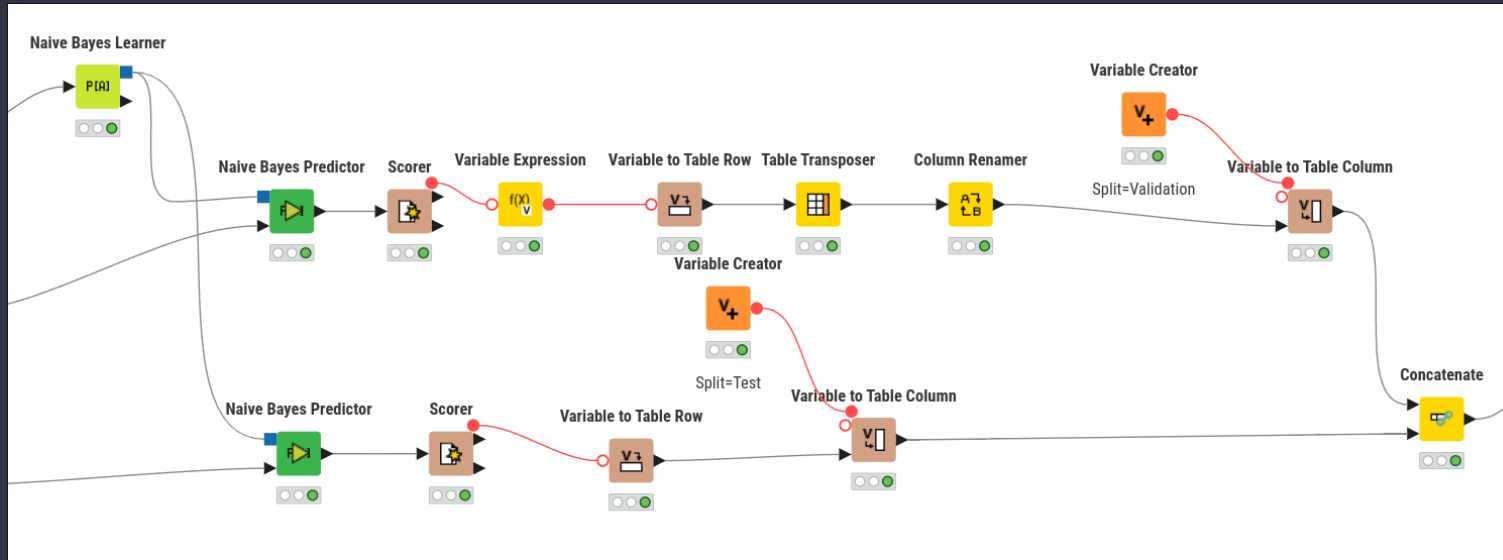
Variable To Table Row



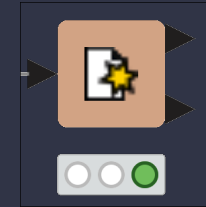
Confidence Interval with Knime - Accuracy

Knime does not have any node for computing the **Wilson Score Interval**.

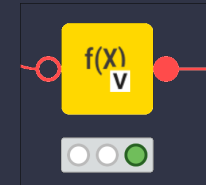
Thus, we need to implement it using the *Expression/Variable Expression* node.



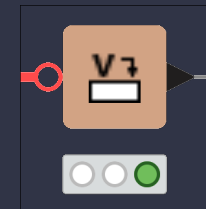
Scorer



Variable Expression



Variable To Table Row



Confidence Interval - Compare models - Different Test Sets

How can we determine if the performance difference between two models is statistically relevant?

Confidence Interval - Compare models - Different Test Sets

How can we determine if the performance difference between two models is statistically relevant?

$$\left(\overbrace{d - z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Lower Bound}}, \overbrace{d + z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Upper Bound}} \right)$$

where:

- e_1 : Error of model 1
- n_1 : Size of test set 1.
- $d = e_1 - e_2$
- e_2 : Error of model 2
- n_2 : Size of test set 2.
- $\hat{\sigma}_d^2 = \frac{e_1*(1-e_1)}{n_1} + \frac{e_2*(1-e_2)}{n_2}$

Confidence Interval - Compare models - Different Test Sets

How can we determine if the performance difference between two models is statistically relevant?

$$\left(\overbrace{d - z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Lower Bound}}, \overbrace{d + z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Upper Bound}} \right)$$

where:

- e_1 : Error of model 1
- n_1 : Size of test set 1.
- $d = e_1 - e_2$
- e_2 : Error of model 2
- n_2 : Size of test set 2.
- $\hat{\sigma}_d^2 = \frac{e_1*(1-e_1)}{n_1} + \frac{e_2*(1-e_2)}{n_2}$

If the **upper bound** is less than 0, **model 2** is better than **model 1**.

Confidence Interval - Compare models - Different Test Sets

How can we determine if the performance difference between two models is statistically relevant?

$$\left(\overbrace{d - z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Lower Bound}}, \overbrace{d + z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Upper Bound}} \right)$$

where:

- e_1 : Error of model 1
- n_1 : Size of test set 1.
- $d = e_1 - e_2$
- e_2 : Error of model 2
- n_2 : Size of test set 2.
- $\hat{\sigma}_d^2 = \frac{e_1*(1-e_1)}{n_1} + \frac{e_2*(1-e_2)}{n_2}$

If the **lower bound** is **greater than 0**, **model 1** is better than **model 2**.

Confidence Interval - Compare models - Different Test Sets

How can we determine if the performance difference between two models is statistically relevant?

$$\left(\overbrace{d - z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Lower Bound}}, \overbrace{d + z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Upper Bound}} \right)$$

where:

- e_1 : Error of model 1
- n_1 : Size of test set 1.
- $d = e_1 - e_2$
- e_2 : Error of model 2
- n_2 : Size of test set 2.
- $\hat{\sigma}_d^2 = \frac{e_1*(1-e_1)}{n_1} + \frac{e_2*(1-e_2)}{n_2}$

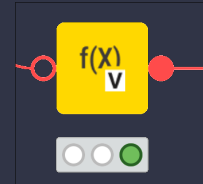
If **0 spans between the lower bound and the upper bound** than the difference is not statistically significative.

Compare models with Knime - Different Test Sets

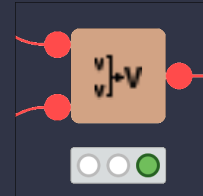
Knime does not have any node for estimating the confidence interval for the difference of two models with different test set.

Thus, we need to implement it using the *Expression/Variable Expression* node.

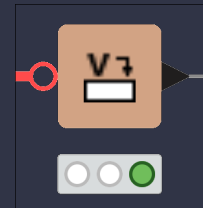
Variable Expression



Merge Variables



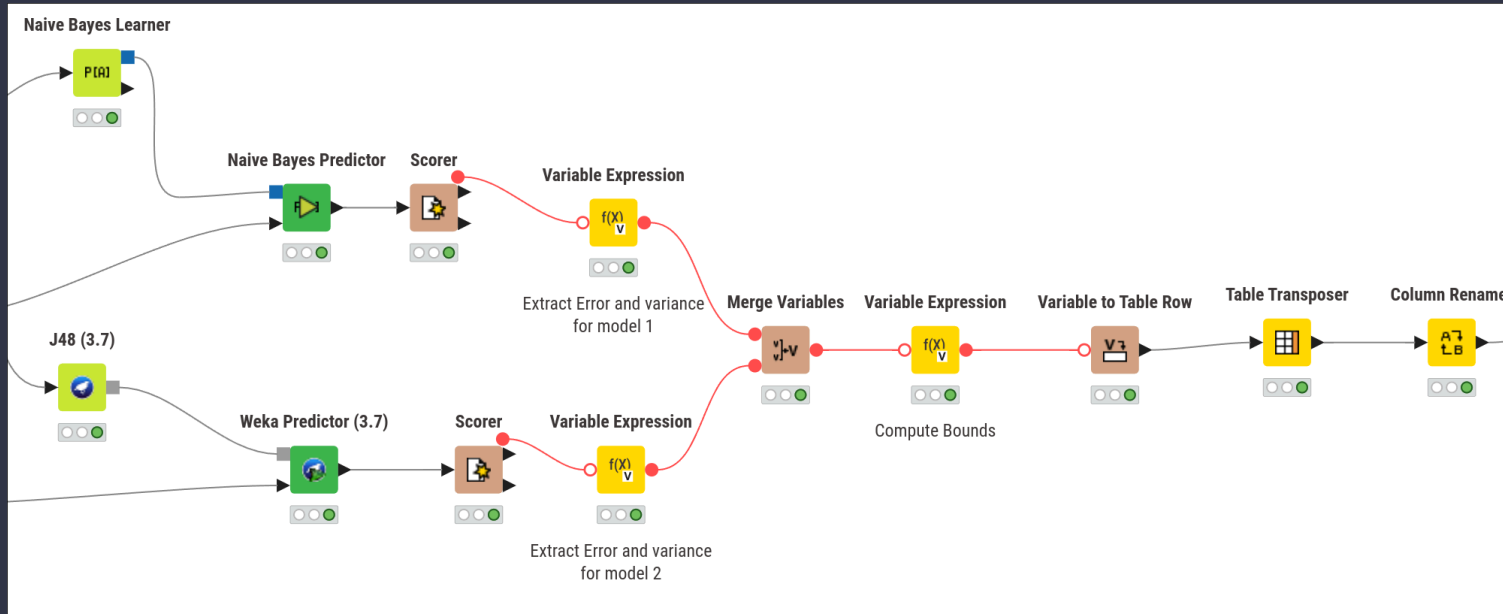
Variable To Table Row



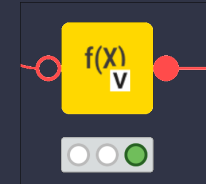
Compare models with Knime - Different Test Sets

Knime does not have any node for estimating the confidence interval for the difference of two models with different test set.

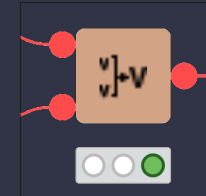
Thus, we need to implement it using the *Expression/Variable Expression* node.



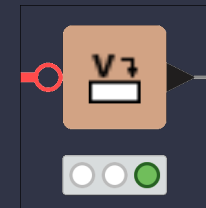
Variable Expression



Merge Variables



Variable To Table Row



Confidence Interval - Compare models - Same Test Set

The **paired t-test** is a statistical test that can be used to compare two models over the same test-set

Confidence Interval - Compare models - Same Test Set

The **paired t-test** is a statistical test that can be used to compare two models over the same test-set

Specifically, this approach can be applied when we use a Cross-Validation technique:

Confidence Interval - Compare models - Same Test Set

The **paired t-test** is a statistical test that can be used to compare two models over the same test-set

Specifically, this approach can be applied when we use a Cross-Validation technique:

$$\left(\bar{d} - t_{1-\frac{\alpha}{2}}^{K-1} \cdot \hat{\sigma}_{d^{CV}}, \bar{d} + t_{1-\frac{\alpha}{2}}^{K-1} \cdot \hat{\sigma}_{d^{CV}} \right)$$

where:

- K : is the number of folds
- $\bar{d} = \frac{1}{K} \sum_{k=1}^K d_k$
- $\hat{\sigma}_{d^{CV}}^2 = \frac{\sum_{k=1}^K (d_k - \bar{d})^2}{K \cdot (K-1)}$
- $t_{1-\frac{\alpha}{2}}^{K-1}$: is the quantile of a Student's t
- $d_k = e_{1k} - e_{2k}$: is the difference of the error of the two models at k-th fold.

Confidence Interval - Compare models - Same Test Set

The **paired t-test** is a statistical test that can be used to compare two models over the same test-set

Specifically, this approach can be applied when we use a Cross-Validation technique:

$$\left(\bar{d} - t_{1-\frac{\alpha}{2}}^{K-1} \cdot \hat{\sigma}_{d^{CV}}, \bar{d} + t_{1-\frac{\alpha}{2}}^{K-1} \cdot \hat{\sigma}_{d^{CV}} \right)$$

where:

- K : is the number of folds
- $\bar{d} = \frac{1}{K} \sum_{k=1}^K d_k$
- $\hat{\sigma}_{d^{CV}}^2 = \frac{\sum_{k=1}^K (d_k - \bar{d})^2}{K \cdot (K-1)}$
- $t_{1-\frac{\alpha}{2}}^{K-1}$: is the quantile of a Student's t
- $d_k = e_{1k} - e_{2k}$: is the difference of the error of the two models at k-th fold.

If the **upper bound** is less than 0, **model 2** is better than **model 1**.

Confidence Interval - Compare models - Same Test Set

The **paired t-test** is a statistical test that can be used to compare two models over the same test-set

Specifically, this approach can be applied when we use a Cross-Validation technique:

$$\left(\bar{d} - t_{1-\frac{\alpha}{2}}^{K-1} \cdot \hat{\sigma}_{d^{CV}}, \bar{d} + t_{1-\frac{\alpha}{2}}^{K-1} \cdot \hat{\sigma}_{d^{CV}} \right)$$

where:

- K : is the number of folds
- $\bar{d} = \frac{1}{K} \sum_{k=1}^K d_k$
- $d_k = e_{1k} - e_{2k}$: is the difference of the error of the two models at k-th fold.
- $\hat{\sigma}_{d^{CV}}^2 = \frac{\sum_{k=1}^K (d_k - \bar{d})^2}{K \cdot (K-1)}$
- $t_{1-\frac{\alpha}{2}}^{K-1}$: is the quantile of a Student's t

If the **lower bound** is **greater** than **0**, **model 1** is better than **model 2**.

Confidence Interval - Compare models - Same Test Set

The **paired t-test** is a statistical test that can be used to compare two models over the same test-set

Specifically, this approach can be applied when we use a Cross-Validation technique:

$$\left(\bar{d} - t_{1-\frac{\alpha}{2}}^{K-1} \cdot \hat{\sigma}_{d^{CV}}, \bar{d} + t_{1-\frac{\alpha}{2}}^{K-1} \cdot \hat{\sigma}_{d^{CV}} \right)$$

where:

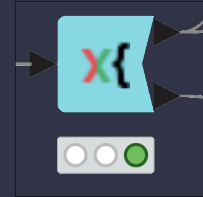
- K : is the number of folds
- $\bar{d} = \frac{1}{K} \sum_{k=1}^K d_k$
- $\hat{\sigma}_{d^{CV}}^2 = \frac{\sum_{k=1}^K (d_k - \bar{d})^2}{K \cdot (K-1)}$
- $t_{1-\frac{\alpha}{2}}^{K-1}$: is the quantile of a Student's t
- $d_k = e_{1k} - e_{2k}$: is the difference of the error of the two models at k-th fold.

If **0 spans between the lower bound and the upper bound** than the difference is not statistically significative.

Compare models with Knime - Same Test Set

The *Paired t-test* node provided by the Knime standard library, implements the test for comparing two dataset with the same test set.

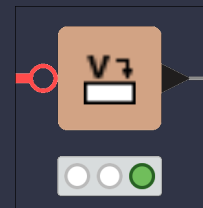
X-Partitioner



X-Aggregator

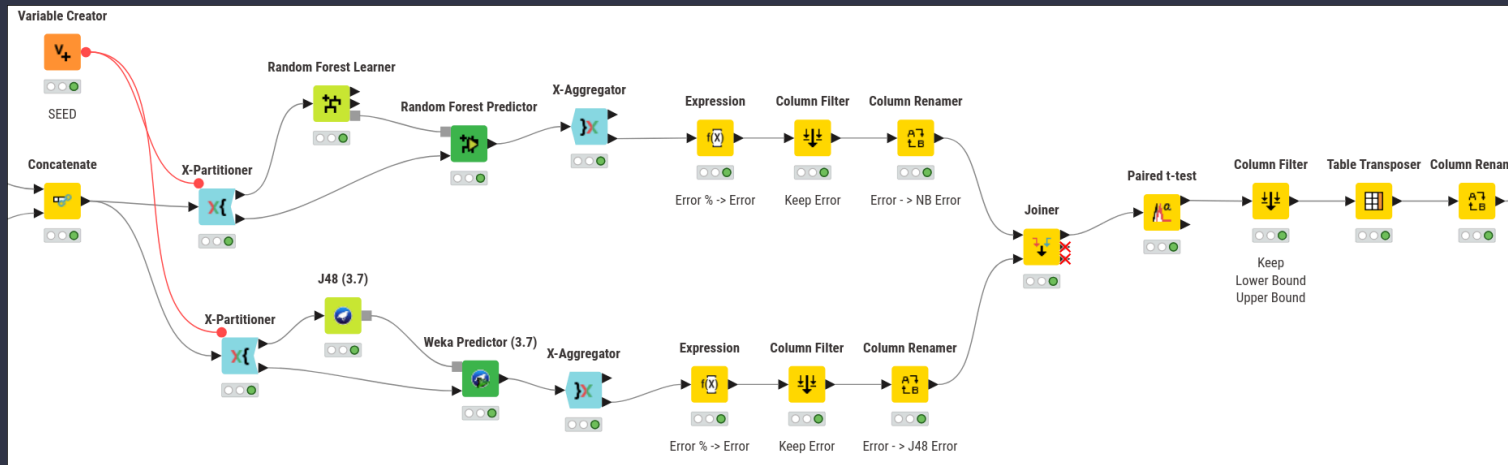


Paired t-test

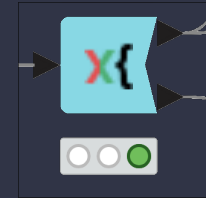


Compare models with Knime - Same Test Set

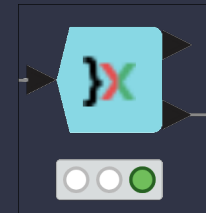
The *Paired t-test* node provided by the Knime standard library, implements the test for comparing two dataset with the same test set.



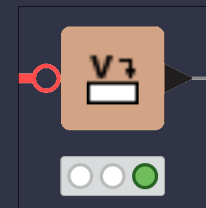
X-Partitioner



X-Aggregator

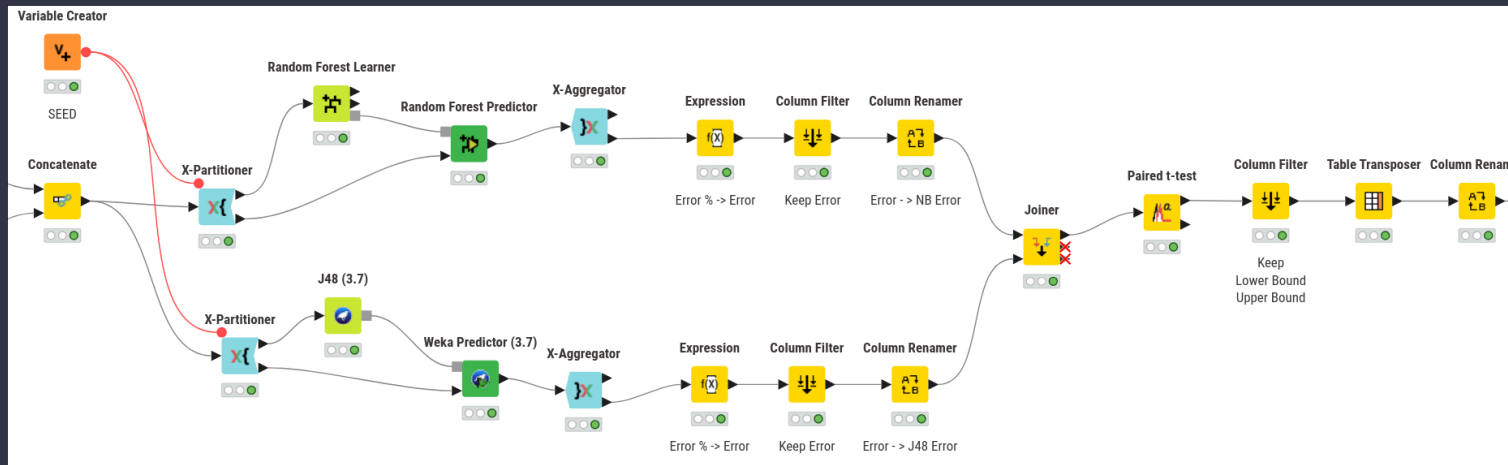


Paired t-test

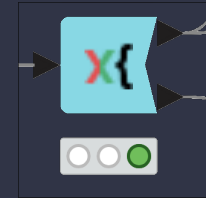


Compare models with Knime - Same Test Set

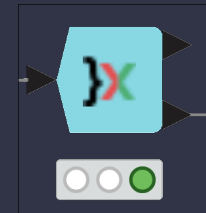
The *Paired t-test* node provided by the Knime standard library, implements the test for comparing two dataset with the same test set.



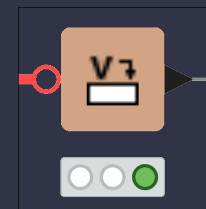
X-Partitioner



X-Aggregator



Paired t-test



In order to perform a **paired t-test** it is fundamental to **fix the seed** of the *X-Partitioner* nodes. This allow us to compare the models on the **same datasets**.

Hands on (30 minutes)

- **Load** the dataset **Churn_yn.xls**
- **Load** the **Churn_yn.xls** dataset.
- **Remove** the columns **State**, **Phone** and **Area Code**.
- **Split** the dataset in **Train**, **Validation** and **Test**
- **Train** a classifier of your choice on the **Train** set
- **Compute** the **Wilson Score Interval** on the **Validation Set** using a *Scorer* and a *Variable Expression*:

$$\left(\frac{\text{acc} + \frac{z_{1-\frac{\alpha}{2}}^2}{2n} - \frac{z_{1-\frac{\alpha}{2}}}{2n} \sqrt{4n \cdot \text{acc} \cdot (1 - \text{acc}) + z_{1-\frac{\alpha}{2}}^2}}{1 + \frac{z_{1-\frac{\alpha}{2}}^2}{n}}, \frac{\text{acc} + \frac{z_{1-\frac{\alpha}{2}}^2}{2n} + \frac{z_{1-\frac{\alpha}{2}}}{2n} \sqrt{4n \cdot \text{acc} \cdot (1 - \text{acc}) + z_{1-\frac{\alpha}{2}}^2}}{1 + \frac{z_{1-\frac{\alpha}{2}}^2}{n}} \right)$$

- $z_{1-\frac{0.05}{2}} = 1.96$
- **Compute** the **error** rate on the **test set**.
- **Show** the results using a *boxplot*



Hands on (20 minutes)

- **Extend** the previous workflow
- **Train** two classifiers of your choice on the **Train** set
- **Evaluate** the two models one on the **test set** and the other on the **validation set**.
- **Compute** the bounds:

$$\left(\overbrace{d - z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Lower Bound}}, \overbrace{d + z_{1-\frac{\alpha}{2}} \hat{\sigma}_d}^{\text{Upper Bound}} \right)$$

where:

- $d = e_1 - e_2$
- $\hat{\sigma}_d^2 = \frac{e_1*(1-e_1)}{n_1} + \frac{e_2*(1-e_2)}{n_2}$
- $z_{1-\frac{0.05}{2}} = 1.96$
- **Show** the results using a *boxplot*
- What can you tell about the performance of the two classifiers?



Hands on (15 minutes)

- **Extend** the previous workflow
- **Merge train** and **validation**
- **Train** two classifiers of your choice using the cross validation technique.
 - *hint: Use the nodes X-Partitioner and X-Aggregator*
 - *hint: Set an identical random seed on each X-Partitioner node so that all models are trained and validated using the exact same data partitions.*
- **Compute** the bound using the *Paired t-test* node.
- **Show** the results using a *boxplot*
- What can you tell about the performance of the two classifiers?

